

Anexo 1: RUTAS Project: Robots for Semantica-based and Autonomous Urban Tourist Centers

Development team

Version: 8.0



Pepper Robots Simulation

Reference Website: https://github.com/awesomebytes/pepper_virtual

Install ROS Kinetic, Gazebo 7 and Ubuntu 16.04:

1. Installing dependencies (How you are writing SUDO, you'll be asked for sudo password)

```
#echo "Installing dependencies (you'll be asked for sudo password)..."
$sudo apt-get install ros-kinetic-pepper-meshes
$sudo apt-get install ros-kinetic-gazebo-ros-control
$sudo apt-get install ros-kinetic-moveit*
$sudo apt-get install ros-kinetic-tf2-sensor-msgs ros-kinetic-ros-control ros-kinetic-ros-controllers
ros-kinetic-gazebo-ros ros-kinetic-gazebo-plugins ros-kinetic-controller-manager python-wstool
ros-kinetic-gazebo*
$sudo apt-get install ros-kinetic-openslam-gmapping
$sudo apt-get install ros-kinetic-navigation
$sudo apt-get install ros-kinetic-depthimage-to-laserscan
$sudo apt-get update && sudo apt-get upgrade
```

2. Creating Pepper ROS workspace at "~/pepper_sim_ws"

```
#esub - cho "Creating Pepper ROS workspace at ~/pepper_sim_ws"
$mkdir -p pepper_sim_ws/src
```

3. Change to directory "src"

```
$cd pepper_sim_ws/src
```

4. Installing packages of Pepper from github

```
$git clone -b correct_chain_model_and_gazebo_enabled https://github.com/awesomebytes/pepper_robot
$git clone -b simulation_that_works https://github.com/awesomebytes/pepper_virtual
$git clone https://github.com/awesomebytes/gazebo\_model\_velocity\_plugin
$git clone https://github.com/pal-robotics/ddynamic\_reconfigure\_python
```

5. Change to directory "pepper_sim_ws"

```
$cd ..
```

6. Program Compile

```
#Program Compile
$catkin_make
```

IMPORTANT: In case it's not working..... Download pepper_robot repository and again compile.

```
#branch correct_chain_model_and_gazebo_enabled
```

```
$https://github.com/awesomebytes/pepper\_robot/tree/correct\_chain\_model\_and\_gazebo\_enabled
```

7. Environment setup: Modify original "bashrc" file

```
#Modify original bashrc
$gedit ~/.bashrc
#Insert nest text, after "source /opt/ros/kinetic/setup.bash"
export ROS_PEPPER_SIM_WS=<path_to_your_pepper_sim_ws_folder>
# <path_to_your_pepper_sim_ws_folder> = For Example: /home/jose/pepper_sim_ws
source $ROS_PEPPER_SIM_WS/devel/setup.bash
```



```
PS1="\[\033{debian_chroot:+($debian_chroot)}\u@h: \w\[\033m"
;;
;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias ls='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${SHELL}" "Terminal: $(history | tail -n1 | sed -e '\''s/^s*[0-9]*\s*//;s/[:&]\s*alert$//'\'' )"'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -o posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

source /opt/ros/melodic/setup.bash
export ROS_PACKAGE_PATH=$HOME/pynaoqi/pynaoqi-python2.7-2.5.5.5-linux64/lib/python2.7/site-packages:$PYTHONPATH
source /opt/ros/melodic/setup.bash
```

Add Pepper real:

NAOqi SDK Installation

1.- Download pynaoqi (preferably version 2.5.5):

<https://www.softbankrobotics.com/emea/en/support/pepper-naoqi-2-9/downloads-softwares/former-versions?os=49&category=108>

/home/jose/pepper_sim_ws/home/jose/pepper_sim_ws/home/jose/pepper_sim_ws/home/jose/pepper_sim_ws/home/jose/pepper_sim_ws

#Extract the file “pynaoqi copy-python2.7-2.5.5.5-linux64” from “pynaoqi-python2.7-2.5.5.5-linux64.tar”

#Create the “pynaoqi” file

\$cd

\$mkdir -p pynaoqi

#Copy the file “pynaoqi copy-python2.7-2.5.5.5-linux64” to “/home/jose/pynaoqi”

2. Environment setup: Modify original “bashrc” file

#Modify original bashrc

\$gedit ~/.bashrc

#Insert nest text, after “source \$ROS_PACKAGE_PATH/devel/setup.bash”

export PYTHONPATH=\$HOME/pynaoqi/pynaoqi-python2.7-2.5.5.5-linux64/lib/python2.7/site-packages:\$PYTHONPATH

Installation of some additional ROS packages

3. Install available bridges between ROS and NAOqi

\$ sudo apt-get install ros-kinetic-driver-base ros-kinetic-move-base-msgs ros-kinetic-octomap
ros-kinetic-octomap-msgs ros-kinetic-humanoid-msgs ros-kinetic-humanoid-nav-msgs

ros-kinetic-camera-info-manager ros-kinetic-camera-info-manager-py

\$ sudo apt-get install ros-kinetic-pepper-.*

Starting the bridge on a real Pepper

4. Verifique Robot IP (exemplo)

<http://192.168.0.162/#/menu/myrobot>

5. Active ROS

\$roscore

6. Connection with the robot pepper

\$roslaunch pepper_bringup pepper_full_py.launch nao_ip:=192.168.0.162 roscore_ip:=http://localhost:11311
network_interface:=wlan0

Install ROS Melodic, Gazebo 9 and Ubuntu 18.04:

1. Installing dependencies (How you are writing SUDO, you'll be asked for sudo password)

```
sudo apt-get install ros-melodic-pepper-meshes
sudo apt-get install ros-melodic-gazebo-ros-control ros-melodic-rqt-joint-trajectory-controller
sudo apt-get install ros-melodic-moveit*
sudo apt-get install ros-melodic-tf2-sensor-msgs ros-melodic-ros-control ros-melodic-ros-controllers
ros-melodic-gazebo-ros ros-melodic-gazebo-plugins ros-melodic-controller-manager python-wstool
ros-melodic-gazebo*
```

2. Creating Pepper ROS workspace at “~/pepper_sim_ws”

```
$mkdir -p pepper_sim_ws/src
```

3. Change to directory “src”

```
$cd pepper_sim_ws/src
```

4. Installing packages of Pepper from github

```
# $git clone -b correct_chain_model_and_gazebo_enabled https://github.com/awesomebytes/pepper_robot
$git clone https://github.com/marco-quiroz/pepper_robot
$git clone -b simulation_that_works https://github.com/awesomebytes/pepper_virtual
$git clone https://github.com/awesomebytes/gazebo_model_velocity_plugin
$git clone https://github.com/pal-robotics/ddynamic_reconfigure_python
```

5. Change to directory “pepper_sim_ws”

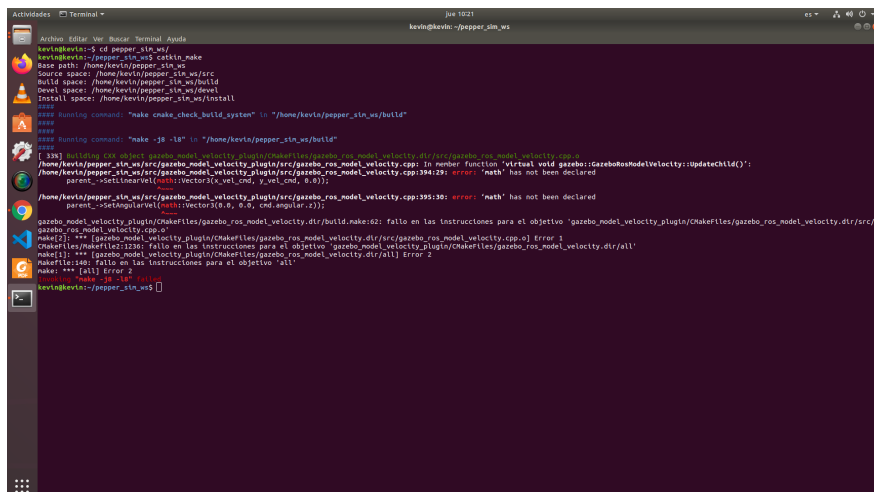
```
$cd ..
```

6. Program Compile

```
$catkin_make
```

7. Correction of compilation mistakes

You will probably get these errors:



```
kevin@kevin:~/pepper_sim_ws$ catkin_make
Base path: /home/kevin/pepper_sim_ws
Source space: /home/kevin/pepper_sim_ws/src
Build space: /home/kevin/pepper_sim_ws/build
Devel space: /home/kevin/pepper_sim_ws/devel
Install space: /home/kevin/pepper_sim_ws/install
####
#### Running command: "make check_build_system" in "/home/kevin/pepper_sim_ws/build"
####
####
#### Running command: "make -j8 -l8" in "/home/kevin/pepper_sim_ws/build"
####
[ 33%] Built target catkin_make
/home/kevin/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp: In member function 'virtual void gazebo::GazeboModelVelocity::UpdateCh1d()':
/home/kevin/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp:394:29: error: 'math' has not been declared
   parent->SetLinearVel(math::Vector3(0.0, 0.0, 0.0));
                           ^~~~~
/home/kevin/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp:395:30: error: 'math' has not been declared
   parent->SetAngularVel(math::Vector3(0.0, 0.0, 0.0).cnd.angular.i);
                           ^~~~~
gazebo_model_velocity_plugin/CMakefiles/gazebo_ros_model_velocity.dir/build.make:62: fallo en las instrucciones para el objetivo 'gazebo_model_velocity_plugin/CMakefiles/gazebo_ros_model_velocity.dir/src/
gazebo_ros_model_velocity.cpp.o'
make[2]: *** [gazebo_model_velocity_plugin/CMakefiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o] Error 1
CMakeFiles/gazebo_model_velocity.dir/build.make:1230: fallo en las instrucciones para el objetivo 'gazebo_model_velocity_plugin/CMakefiles/gazebo_ros_model_velocity.dir/all'
make[1]: *** [gazebo_model_velocity_plugin/CMakefiles/gazebo_ros_model_velocity.dir/all] Error 2
makefile:1230: fallo en las instrucciones para el objetivo 'all'
make: *** [all] Error 2
kevin@kevin:~/pepper_sim_ws$
```

7.1 Error:

Error happened in the file: "gazebo_ros_model_velocity.cpp", specifically in the lines "394" y "395" of the code.

To go to this file, you have to run the next command:

```
$cd /home/dslam/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/  
$gedit gazebo_ros_model_velocity.cpp
```

In both lines of code a change must be made, because we are working with Gazebo 9, then we must update the syntax of the code so that it compiles without errors. So initially the lines have the following form

AN in initial pose Yaw

Line 394: parent_>SetLinearVel(math::Vector3(x_vel_cmd, y_vel_cmd, 0.0));

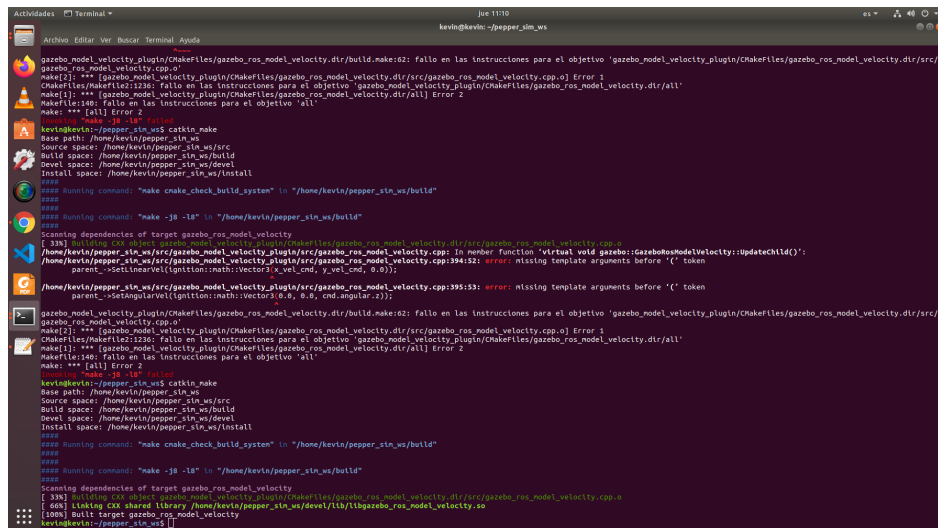
Line 395: parent_>SetAngularVel(math::Vector3(0.0, 0.0, cmd.angular.z));

Making the changes, the lines should be as follows:

Line 394: parent_>SetLinearVel(ignition::math::Vector3d(x_vel_cmd, y_vel_cmd, 0.0));

Line 395: parent_>SetAngularVel(ignition::math::Vector3d(0.0, 0.0, cmd.angular.z));

So after saving the file changes, you should proceed to compile again and the error should no longer exist. As shown in the following figure:



```
gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/build.make:62: fallo en las instrucciones para el objetivo 'gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o'
make[1]: *** [gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/build.make:62: gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o] Error 1
make[1]: *** [gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/build.make:62: gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/all] Error 1
make: *** [all] Error 2

make -j8 -l8 failed
make: *** [make -j8 -l8] Error 2

kinetic@pepper:~/pepper_sim_ws$ catkin_make
Base path: /home/kinetic/pepper_sim_ws
Source space: /home/kinetic/pepper_sim_ws/src
Build space: /home/kinetic/pepper_sim_ws/build
Devel space: /home/kinetic/pepper_sim_ws/devel
Install space: /home/kinetic/pepper_sim_ws/install
####
#### Running command "make cmake_check_build_system" in "/home/kinetic/pepper_sim_ws/build"
####
####
#### Running command "make -j8 -l8" in "/home/kinetic/pepper_sim_ws/build"
####
Scanning dependencies of target gazebo_ros_model_velocity
[ 33%] Building CXX object gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o
/home/kinetic/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp: In member function 'virtual void gazebo::GazeboRosModelVelocity::updateVelCmd()':
/home/kinetic/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp:394:52: error: missing template arguments before '(' token
parent->SetLinearVel(ignition::math::Vector3(x_vel_cmd, y_vel_cmd, 0.0));
                        ^
/home/kinetic/pepper_sim_ws/src/gazebo_model_velocity_plugin/src/gazebo_ros_model_velocity.cpp:395:53: error: missing template arguments before '(' token
parent->SetAngularVel(ignition::math::Vector3(0.0, 0.0, cmd.angular.z));
                        ^
make: *** [gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/build.make:62: gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o] Error 1
make[1]: *** [gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/build.make:62: gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/all] Error 1
make: *** [all] Error 2

make -j8 -l8 failed
make: *** [make -j8 -l8] Error 2

kinetic@pepper:~/pepper_sim_ws$ catkin_make
Base path: /home/kinetic/pepper_sim_ws
Source space: /home/kinetic/pepper_sim_ws/src
Build space: /home/kinetic/pepper_sim_ws/build
Devel space: /home/kinetic/pepper_sim_ws/devel
Install space: /home/kinetic/pepper_sim_ws/install
####
#### Running command "make cmake_check_build_system" in "/home/kinetic/pepper_sim_ws/build"
####
####
#### Running command "make -j8 -l8" in "/home/kinetic/pepper_sim_ws/build"
####
Scanning dependencies of target gazebo_ros_model_velocity
[ 33%] Building CXX object gazebo_model_velocity_plugin/CMakeFiles/gazebo_ros_model_velocity.dir/src/gazebo_ros_model_velocity.cpp.o
[ 66%] Linking CXX shared library /home/kinetic/pepper_sim_ws/devel/lib/libgazebo_ros_model_velocity.so
[100%] Built target gazebo_ros_model_velocity
kinetic@pepper:~/pepper_sim_ws$
```

8. Environment setup: Modify original "bashrc" file

#Modify original bashrc

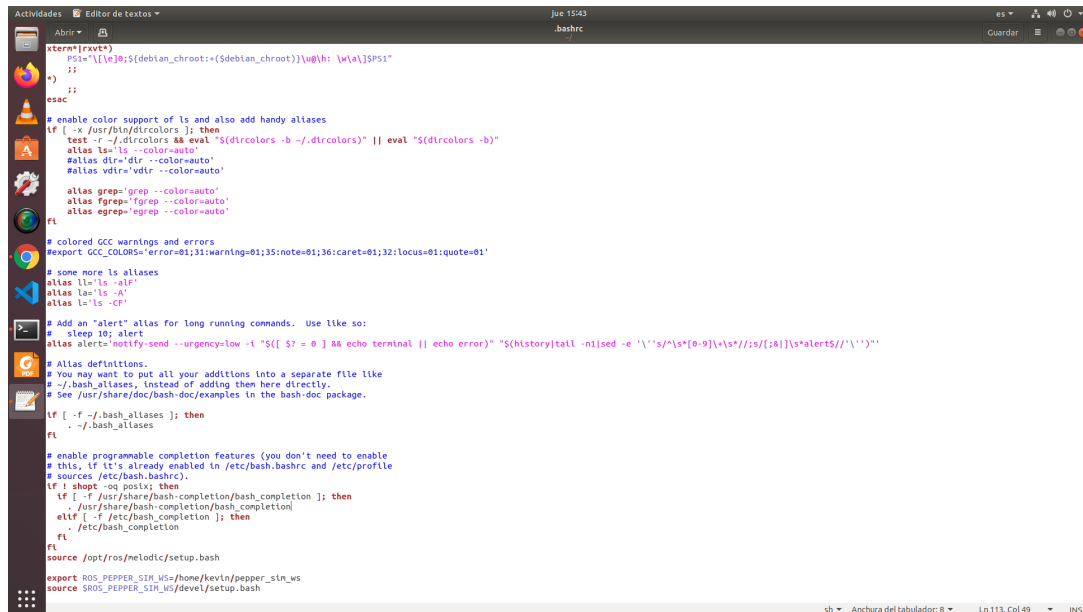
```
$gedit ~/.bashrc
```

#Insert next text, after "source /opt/ros/melodic/setup.bash"

```
export ROS_PEPPER_SIM_WS=<path_to_your_pepper_sim_ws_folder>
```

```
# <path_to_your_pepper_sim_ws_folder> = For Example: /home/jose/pepper_sim_ws
```

```
source $ROS_PEPPER_SIM_WS/devel/setup.bash
```



```
PS1='\[\e\];$(debian_chroot:+$(debian_chroot))\u@h: \w\]\$PS1'
;;
;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'
    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'

# colored GCC warnings and errors
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias ls='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&]\s*alert$//'\''")'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

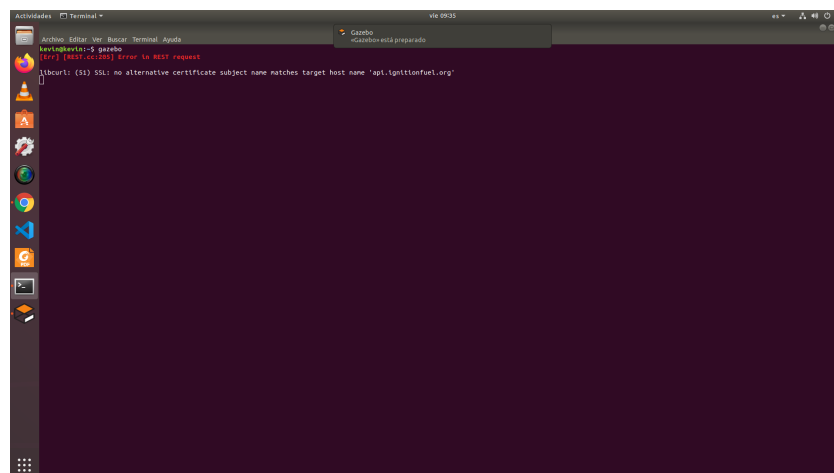
source /opt/ros/melodic/setup.bash
export ROS_PEPPER_STIR_WS=/home/kevin/pepper_stir_ws
source $ROS_PEPPER_STIR_WS/devel/setup.bash
```

9. Prove the Gazebo operation

\$gazebo

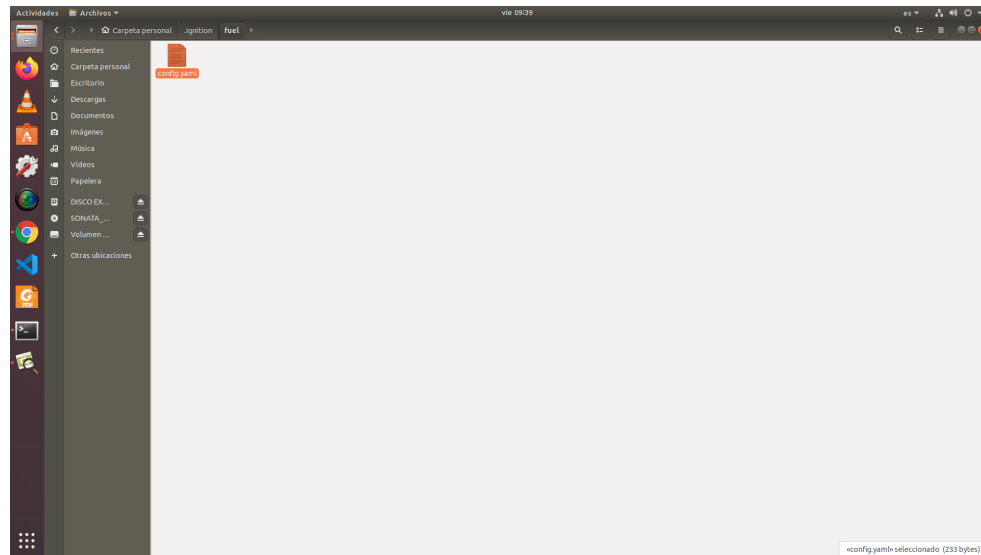
9.1 Correction of Gazebo mistake

You will probably get this error:

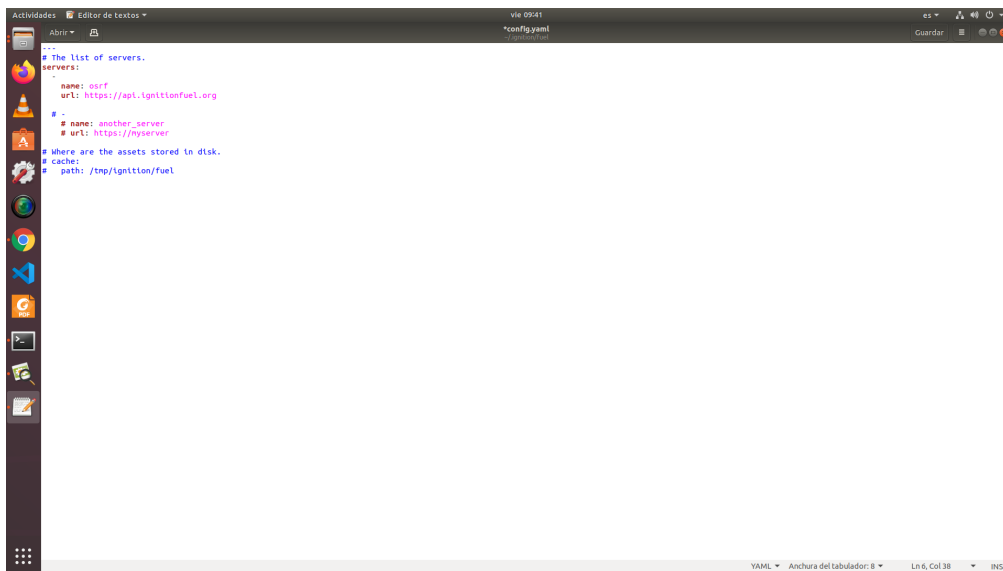


This can be solved as follows:

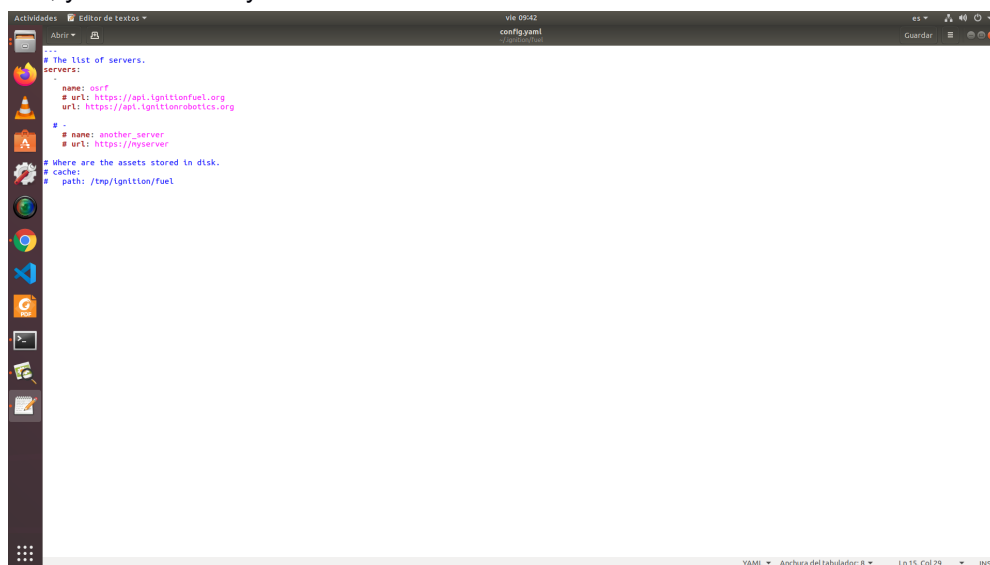
- First, you must go to your "Personal Folder" or "Home" directory.
- Second, you must press the "Ctrl + H" keys to make all the files visible.
- Third, you must look for the directory "ignition" in those files that are already visible, and once inside "ignition", you must enter the directory "fuel", so that you find a file named "config.yaml"



- Fourth, you should open the file "config.yaml" and find the following:

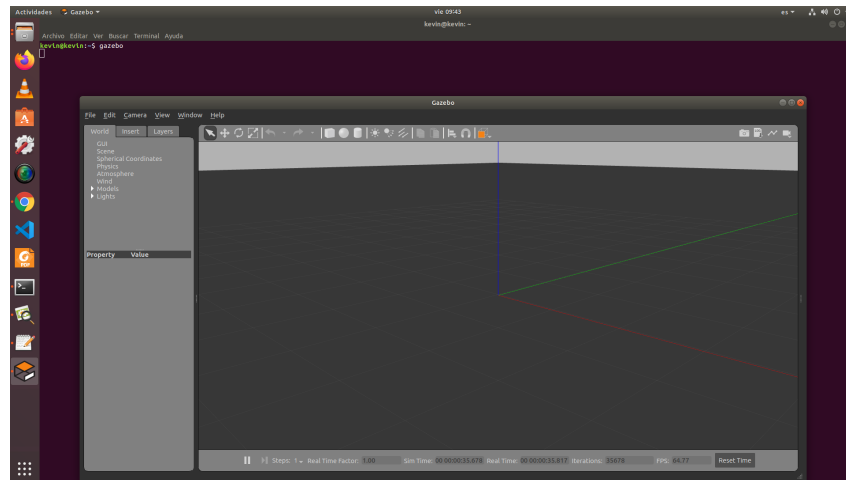


- Fifth, you must modify and save the file so that it looks like this:



Once you have modified and saved the changes, there is nothing left but to launch Gazebo

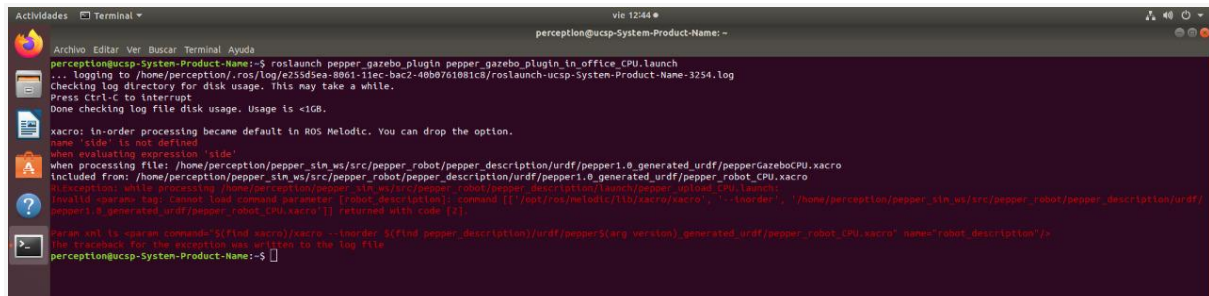
again from the terminal, and the previous error should no longer appear, and Gazebo will run naturally, as shown below:



NOTE:

The package: `$git clone -b correct_chain_model_and_gazebo_enabled`

https://github.com/awesomebytes/pepper_robot is not used because the current version of Ubuntu 18.04 gives us the following error:



To solve this error we go to the “pepper1.0_generated_urdf” folder:

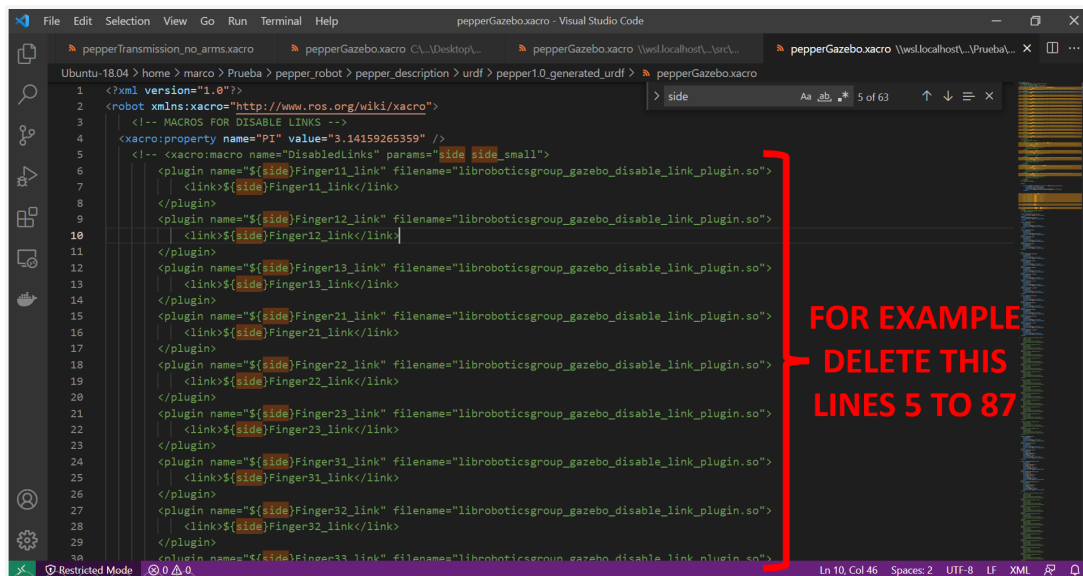
```
$ cd
```

```
/home/jose/pepper_sim_ws/src/pepper_robot/pepper_description/urdf/pepper1.0_generated_urdf
```

and remove comments containing the word “side” in the following files:

- pepperGazebo.xacro
- pepperGazeboCPU.xacro
- pepperGazeboCPU_no_arms.xacro
- pepperTransmission.xacro
- pepperTransmission_no_arms.xacro

For example in the file “pepperGazebo.xacro”:



Add Pepper real:

NAOqi SDK Installation

1.- Download pynaoqi (preferably version 2.5.5 of Python):

<https://www.softbankrobotics.com/emea/en/support/pepper-naoqi-2-9/downloads-softwares/former-versions?os=49&category=108>

#Extract the file “pynaoqi copy-python2.7-2.5.5.5-linux64” from “pynaoqi-python2.7-2.5.5.5-linux64.tar”

#Create the “pynaoqi”

\$cd

\$mkdir -p pynaoqi

#Copy the file “pynaoqi copy-python2.7-2.5.5.5-linux64” to “/home/jose/pynaoqi”

2. Environment setup: Modify original “bashrc” file

#Modify original bashrc

\$gedit ~/.bashrc

#Insert next text, after “source \$ROS_PEPPER_SIM_WS/devel/setup.bash”

export PYTHONPATH=\$HOME/pynaoqi/pynaoqi-python2.7-2.5.5.5-linux64/lib/python2.7/site-packages:\$PYTHONPATH

Installation of some additional ROS packages

3. Install available bridges between ROS and NAOqi

```
$ sudo apt-get install ros-melodic-driver-base ros-melodic-move-base-msgs ros-melodic-octomap  
ros-melodic-octomap-msgs ros-melodic-map-server ros-melodic-camera-info-manager  
ros-melodic-camera-info-manager-py ros-melodic-rgbd-launch
```

```
$ sudo apt-get install ros-melodic-pepper-.*
```

4. Install package "naoqi_bridge" and "naoqi_bridge_msgs" :

```
$ cd pepper_sim_ws/src
```

```
$ git clone https://github.com/ros-naoqi/naoqi_bridge.git
```

```
$ git clone https://github.com/ros-naoqi/naoqi_bridge_msgs.git
```

```
$ cd ..
```

```
$ catkin_make
```

Starting the bridge on a real Pepper

5. Verifique Robot IP (exemplo)

<http://192.168.0.162/#/menu/myrobot>

6. Active ROS

```
$roscore
```

7. Connection with the robot pepper

```
$roslaunch pepper_bringup pepper_full_py.launch nao_ip:=192.168.0.162 roscore_ip:=http://localhost:11311  
network_interface:=wlan0
```

Environment Gazebo: Museum and Pepper robot:

Enviroment 01: Create world in gazebo or Copy the “museum” file in the “Worlds” folder
\$gazebo

#Search for "ISCA Museum" in the existing worlds of the gazebo within the "insert" tab and save as
“museum.world”.

#Put “museum.world” inside this folder “worlds”:

~/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/worlds

#Modify original launch

#Open through the terminal in folder “launch”: /pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/launch

\$gedit pepper_gazebo_plugin_in_office_CPU.launch

#Here:

<arg name="world_name" value="\$(find pepper_gazebo_plugin)/worlds/simple_office_with_people.world"/>

Modify:

<arg name="world_name" value="\$(find pepper_gazebo_plugin)/worlds/museum.world"/>

Save as : **pepper_gazebo_plugin_museum.launch**

Enviroment 02: Used world with museum-persons-robots in gazebo:

Copy the “museum_with_persons_robots” file in the “Worlds” folder to

~/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/worlds

#Create new model in ~/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/models

- Make a copy of the folder: “citizen_extras_female_03” and rename: “citizen_extras_female_04”
- Make a copy of the folder: “citizen_extras_male_03” and rename: “citizen_extras_male_04”

Copy “pepper_gazebo_plugin_museum_persons_robots.launch” launch in

~/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/launch

Enviroment 03: Pepper working in Dynamic Environment (people) with **ACTORS**, but this only is possible in
ROS MELODIC.

#Copy from directory “launch” in “Pepper Tutorial V8” to your directory “launch” in

“pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/launch” next file:

“pepper_gazebo_plugin_museum_with_people_moving.launch”

#Copy from directory “worlds” in “Pepper Tutorial V8” to your directory “worlds” in

“pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin” next file:

“museum_with_people_moving.world”

#Download from github the package to solve the collision in actors, download in “pepper_sim_ws/src”

\$git clone https://github.com/JiangweiNEU/actor_collisions.git

#Then, create a new directory named “build” in “pepper_sim_ws/src/actor_collisions” directory:

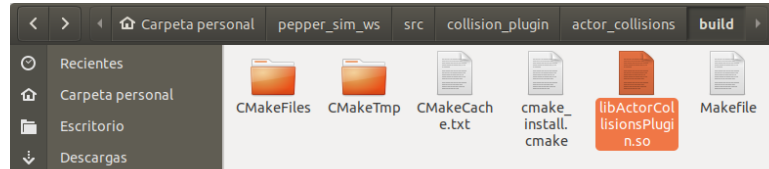
\$mkdir build

#Enter or joint to “build” directory:

\$cd build

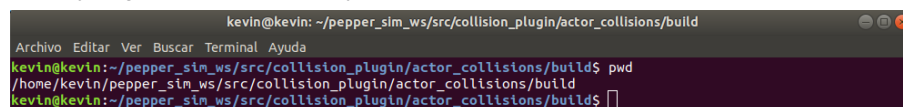
```
#Compile the files downloaded from github
$cmake ..
$make
```

#Ok, now you have to go to the directory where is “libActorCollisionsPlugin.so”. That is in the build directory that you have created a moment ago.



#Then, click right, and select option “Open in a terminal”, and write next:
\$pwd

#Copy the path that you got, for instance in my case is:



#Finally, replace this path in the lines 467, 626, 765, 903, 1038, 1173, 1300, 1451, 1578, 1704, 1838, 1964, 2090, 2216, 2342, 2484 from our file “museum_with_people_moving.world”.

```
<!-- ##### -->
<!-- Actor One -->
<actor name="actor1">
  <plugin name="actor_collisions_plugin" filename="/home/kevin/pepper_sim_ws/src/collision_plugin/actor_collisions/build/libActorCollisionsPlugin.so">
    <scaling collision="LHtpJoint_LeftUpLeg_collision" scale="
      0.01
      0.001
      0.001
    "/>
</actor>
```

Enviroment 04: Pepper working in Dynamic Environment (people) with **AGENTS CLUSTERS**, but this only is possible in **ROS MELODIC**.

#Install from Github the next repository in your “src” file from your workspace.
\$git clone https://github.com/srl-freiburg/pedsim_ros

#Now, do this
\$cd pedsim_ros

#Now, do this
\$git submodule update --init --recursive

#Now, do this
\$cd ../../

#You must compile in your workspace “pepper_sim_ws”:
\$catkin_make

#Copy from directory “launch” located in “Pepper Tutorial V8” to your directory “launch” in “pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/launch” next file:
“pepper_gazebo_plugin_people_moving_randomly_museum.launch”

#Copy from directory “worlds” located in “Pepper Tutorial V8” to your directory “worlds” in “pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/worlds” next file:
“museum_for_agents_clusters.world”

#Copy from directory “Files to Dynamic World” located in “Pepper Tutorial V8” to your directory “launch” in “pepper_sim_ws/src/pedsim_ros/pedsim_simulator/launch” next files:

“simulator_working_in_museum.launch”

“simple_pedestrians_working_in_museum.launch”

#Copy from directory “Files to Dynamic World” located in “Pepper Tutorial V8” to your directory “scenarios” in “pepper_sim_ws/src/pedsim_ros/pedsim_simulator/scenarios” next file:

“social_contexts_working_in_museum.xml”

#You can launch world in gazebo with next command in your terminal:

\$ roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_people_moving_randomly_museum.launch

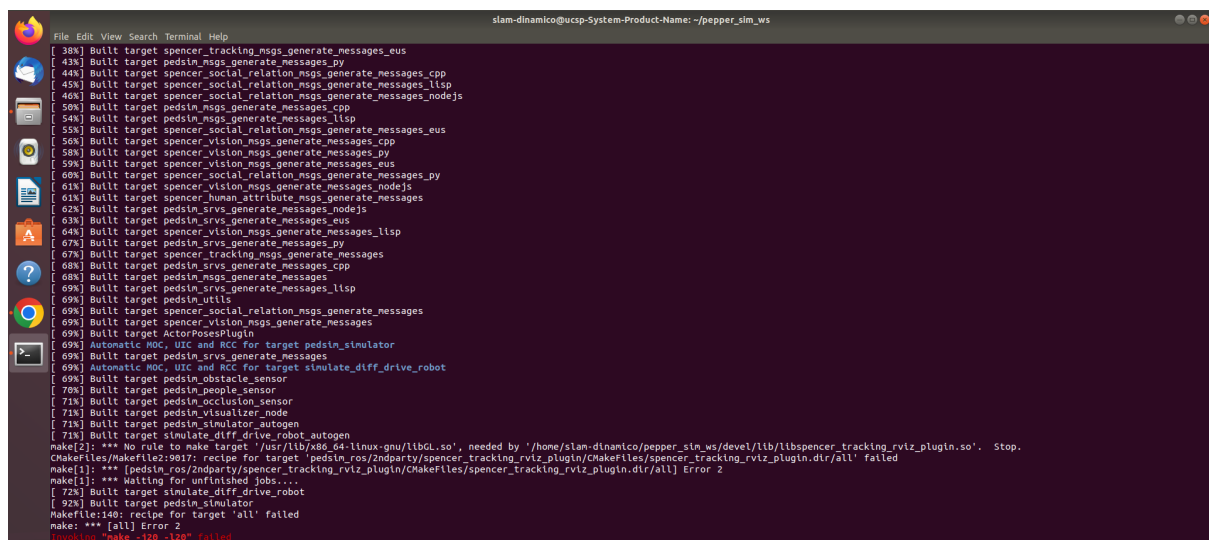
#You can launch rviz and people (agents clusters) with next command in your terminal:

\$ roslaunch pedsim_simulator simple_pedestrians_working_in_museum.launch

Problems that may be present when running the pedestrian simulator

#Example:

No rule to make target '/usr/lib/x86_64-linux-gnu/libGL.so', needed by '/home/slam-dinamico/pepper_sim_ws/devel/lib/libspencer_tracking_rviz_plugin.so'. Stop.



```
slam-dinamico@ucsp-System-Product-Name: ~/pepper_sim_ws
[ 38%] Built target spencer_tracking_msgs_generate_messages_eus
[ 43%] Built target pedsim_msgs_generate_messages_py
[ 44%] Built target spencer_social_relation_msgs_generate_messages_cpp
[ 45%] Built target spencer_social_relation_msgs_generate_messages_lisp
[ 46%] Built target spencer_social_relation_msgs_generate_messages_nodejs
[ 50%] Built target pedsim_msgs_generate_messages_cpp
[ 54%] Built target pedsim_msgs_generate_messages_lisp
[ 55%] Built target spencer_social_relation_msgs_generate_messages_eus
[ 56%] Built target spencer_vision_msgs_generate_messages_cpp
[ 58%] Built target spencer_vision_msgs_generate_messages_py
[ 59%] Built target spencer_vision_msgs_generate_messages_eus
[ 60%] Built target spencer_social_relation_msgs_generate_messages_py
[ 61%] Built target spencer_vision_msgs_generate_messages_nodejs
[ 61%] Built target spencer_human_attribute_msgs_generate_messages
[ 62%] Built target pedsim_srvs_generate_messages_nodejs
[ 63%] Built target pedsim_srvs_generate_messages_eus
[ 64%] Built target spencer_vision_msgs_generate_messages_lisp
[ 67%] Built target pedsim_srvs_generate_messages_py
[ 67%] Built target spencer_tracking_msgs_generate_messages
[ 68%] Built target pedsim_msgs_generate_messages_cpp
[ 68%] Built target pedsim_msgs_generate_messages_lisp
[ 69%] Built target pedsim_srvs_generate_messages_lisp
[ 69%] Built target pedsim_utils
[ 69%] Built target spencer_social_relation_msgs_generate_messages
[ 69%] Built target spencer_vision_msgs_generate_messages
[ 69%] Built target ActorPosesPlugin
[ 69%] Automatic MOC, UIC and RCC for target pedsim_simulator
[ 69%] Built target pedsim_srvs_generate_messages
[ 69%] Automatic MOC, UIC and RCC for target simulate_diff_drive_robot
[ 69%] Built target pedsim_obstacle_sensor
[ 70%] Built target pedsim_people_sensor
[ 71%] Built target pedsim_occlusion_sensor
[ 71%] Built target pedsim_visualizer_node
[ 71%] Built target pedsim_simulator_autogen
[ 71%] Built target simulate_diff_drive_robot_autogen
make[2]: *** No rule to make target '/usr/lib/x86_64-linux-gnu/libGL.so', needed by '/home/slam-dinamico/pepper_sim_ws/devel/lib/libspencer_tracking_rviz_plugin.so'. Stop.
CMakeFiles/Makefile2:9817: recipe for target 'pedsim_ros/2ndparty/spencer_tracking_rviz_plugin/CMakeFiles/spencer_tracking_rviz_plugin.dir/all' failed
make[1]: *** [pedsim_ros/2ndparty/spencer_tracking_rviz_plugin/CMakeFiles/spencer_tracking_rviz_plugin.dir/all] Error 2
make[1]: *** Waiting for unfinished jobs....
[ 72%] Built target simulate_diff_drive_robot
[ 92%] Built target pedsim_simulator
Makefile:140: recipe for target 'all' failed
make: *** [all] Error 2
make[1]: *** [make_job -120] failed
```

#To solve this problem, we must first make sure that all the linux drivers are installed:

\$ sudo ubuntu-drivers autoinstall

#Then, do this:

\$ cd /usr/lib/x86_64-linux-gnu/

#Finally, do this:

\$ sudo ln -sf libGL.so.1.0.0 libGL.so

#And compile again:

\$ cd

\$ cd pepper_sim_ws/

\$ catkin_make

Basic Operation:

Basic Launch in ROS:

#Basic environment

```
$roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_in_office_CPU.launch
```

#Included museum and robot in environments

```
$roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_museum.launch
```

#Included more, persons and robots in environments

```
$roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_museum_persons_robots.launch
```

#Included Pepper working in crowded dynamic environment (people)

```
$roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_museum_with_people_moving.launch
```

#It possible see all sensor the pepper robot in RVIZ:

```
$rosviz rviz -d `rospack find pepper_gazebo_plugin`/config/pepper_sensors.rviz
```

Control robot:

#Pepper robot direction and speed control with RQT

```
#/robot steering: pepper/cmd_vel
```

```
$rosviz rqt_robot_steering rqt_robot_steering
```

#/Pepper robot body control

```
$rosviz rqt_joint_trajectory_controller rqt_joint_trajectory_controller
```

#Manual control the Pepper robot direction and speed

```
$ rostopic pub /pepper/cmd_vel geometry_msgs/Twist "linear:
  x: 0.5
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0" -r 10
```

#Manual control the Pepper robot direction and speed with C++

```
# In pepper_sim_ws/src/
```

```
$catkin_create_pkg random_pepper_driver roscpp std_msgs
```

```
#In pepper_sim_ws/src/random_pepper_driver, Copy and Run "random_driver.cpp"
```

```
#The file "random_driver.cpp" is in the "pepper_virtual/pepper_gazebo_plugin/scripts" folder
```

```
#Modify e add in CMakeLists.txt (in ~/pepper_sim_ws/src/random_husky_driver)
```

```
add_executable(random_driver random_driver.cpp)
```

```
target_link_libraries(random_driver ${catkin_LIBRARIES})
```

```
#Compiled:
```

```
$cd ~/pepper_sim_ws
```

```
$catkin_make
```

```
#Run:
```

```
$rosviz random_pepper_driver random_driver
```

#Control robot with joy-ROS (Is recommend used Xbox [Joystick](#) with USB connection):

```
#Install:
```

```
#Configuring: http://wiki.ros.org/joy/Tutorials/ConfiguringALinuxJoystick
```

```
# Install joy:
```

```
$sudo apt-get install ros-melodic-joy
```

to identify the joystick peripheral (jsX)

```
$ ls /dev/input/
```

Start node:

```
$rosparam set joy_node/dev "/dev/input/jsX"
```

```
$roslaunch joy joy_node
```

#in another tab of the terminal, you check the operation of the Joystick

```
$rostopic echo joy
```

Copy and Run "joy_pepper.py" in

```
~/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/scripts/
```

#The file "joy_pepper.py" is in the "scripts" folder

2D Map

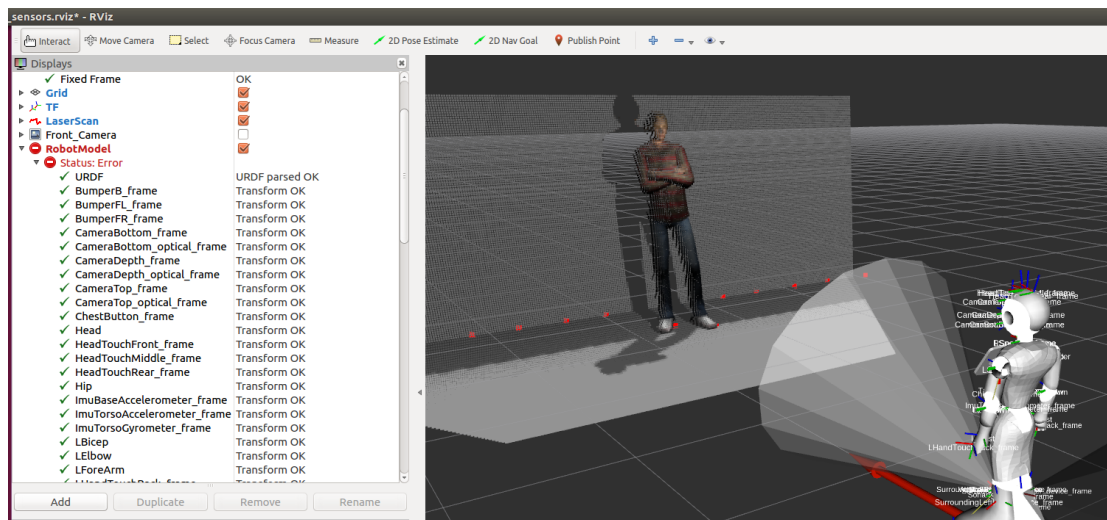
#install

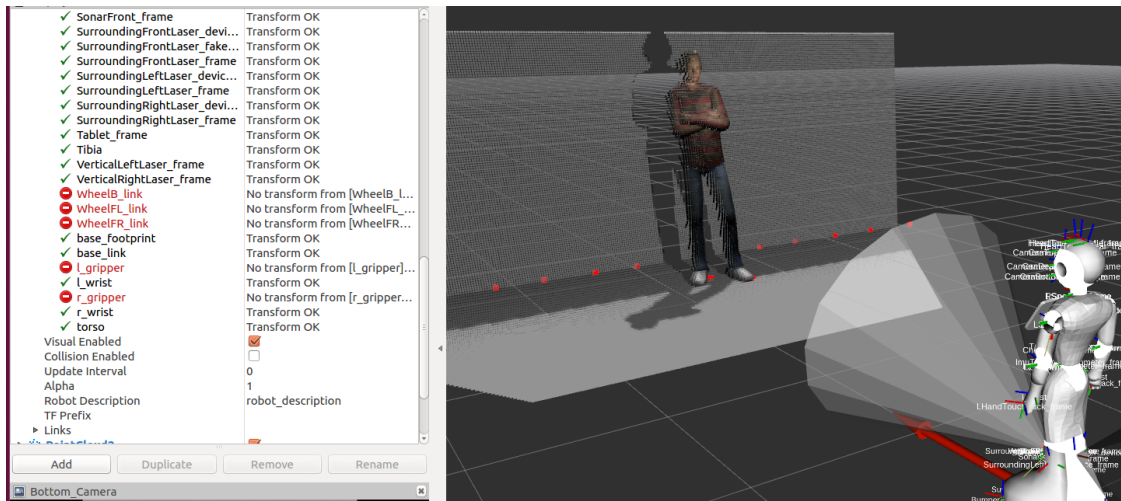
```
$sudo apt-get install python-pygame
```

#Run 2D map with Odometria and random_driver.cpp

```
$roslaunch pepper_gazebo_plugin odom_graph_test.launch
```

Rviz Model Robot Error:





To eliminate problems in these topics: “WheelB_link”, “WheelFL_link” and “WheelFR_link”. It’s necessary to modify the “pepper_robot_CPU.xacro” and “pepper_robot.xacro” files.

These files are located:

/pepper_sim_ws/src/pepper_robot-pepper_correct_chain_model_and_gazebo_enabled/pepper_description/urdf/pepper1.0_generated_urdf/

in “pepper_robot_cpu.xacro” file, it is necessary to comment line (blue):

```
<xacro:include filename="pepper_arms.xacro" />
<xacro:include filename="pepper_sensors.xacro" />
<!-- <xacro:include filename="pepper_fingers.xacro" /> -->
<!-- <xacro:include filename="pepper_wheels.xacro" /> -->
<xacro:include filename="pepperGazeboCPU.xacro" />
```

To eliminate problems in these topics: “l_gripper” and “r_gripper”. It’s necessary to modify the “pepper_arms.xacro” file.

in this file, it is necessary to comment line: (blue):

```
<!-- joint name="LHand" type="revolute">
  <parent link="l_wrist"/>
  <child link="l_gripper"/>
  <origin rpy="0 0 0" xyz="0.025 0 0"/>
  <axis xyz="1.0 0 0"/>
  <limit effort="0.144" lower="0.02" upper="0.98" velocity="12.56"/>
</joint-->
<!-- link name="l_gripper">
  <inertial>
    <mass value="2e-06"/>
    <inertia ixx="1.1e-09" ixy="0" ixz="0" iyy="1.1e-09" iyz="0" izz="1.1e-09"/>
  </inertial>
</link-->
```

```

<!-- joint name="RHand" type="revolute">
  <parent link="r_wrist"/>
  <child link="r_gripper"/>
  <origin rpy="0 0 0" xyz="0.025 0 0"/>
  <axis xyz="1 0 0"/>
  <limit effort="0.144" lower="0.02" upper="0.98" velocity="12.56"/>
</joint>
<link name="r_gripper">
  <inertial>
    <mass value="2e-06"/>
    <inertia ixx="1.1e-09" ixy="0" ixz="0" iyy="1.1e-09" iyz="0" izz="1.1e-09"/>
  </inertial>
</link-->

```

Localization and SLAM

SLAM 2D - Gmapping:

1. (Just do it once): Installing Gmapping - Part 1

```
$sudo apt-get install ros-melodic-openslam-gmapping
```

2. (Just do it once): Installing Gmapping - Part 2

```

$cd pepper_sim_ws/src
$git clone https://github.com/ros-perception/slam_gmapping.git
$cd ..
$catkin_make

```

3. (Do it each time that you work): Modifying Parameters in Gmapping

```

$cd ..
rosparam set slam_gmapping/particles 160
rosparam set slam_gmapping/xmin -10
rosparam set slam_gmapping/ymin -10
rosparam set slam_gmapping/xmax 10
rosparam set slam_gmapping/ymax 10
rosparam set slam_gmapping/srr 0.01
rosparam set slam_gmapping/srt 0.02
rosparam set slam_gmapping/str 0.01
rosparam set slam_gmapping/stt 0.02
rosparam set slam_gmapping/linearUpdate 0.2

```

```
rosparam set slam_gmapping/angularUpdate 0.2
rosparam set slam_gmapping/temporalUpdate 0.5
```

4. (Do it each time that you work): Running SLAM with Gmapping - First option is with “/pepper/laser_2” node

```
$ rosrunc gmapping slam_gmapping scan:=/pepper/laser_2
```

5. (Do it each time that you work): Running SLAM with Gmapping - Second option with “/pepper/hokuyo_scan” node

```
$ rosrunc gmapping slam_gmapping scan:=/pepper/hokuyo_scan
```

6. (Do it each time that you work): Used to starting SLAM Gmapping with map in rviz (in Display Map, select topic “/map”)

```
$roslaunch rviz rviz -d `rospack find pepper_gazebo_plugin`/config/pepper_sensors_map.rviz
```

```
$roslaunch rviz rviz -d `rospack find pepper_gazebo_plugin`/config/pepper_sensors.rviz
```

7. (Just do it once): Installing “map server”-

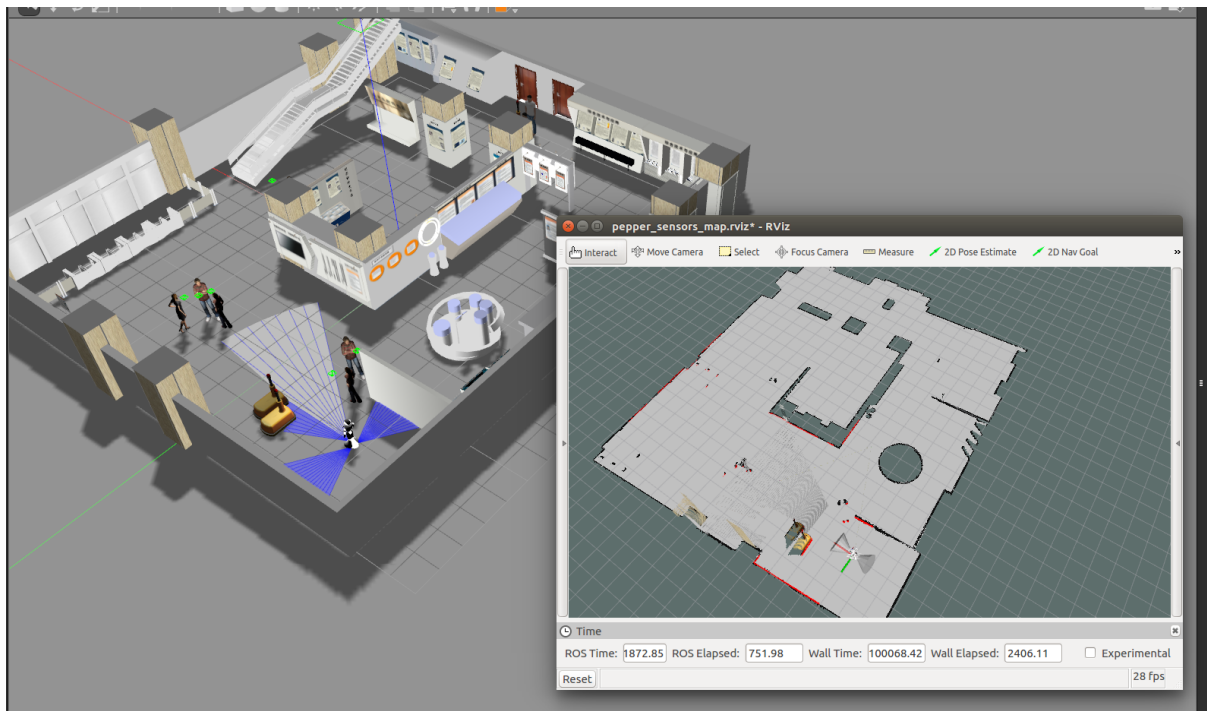
```
$sudo apt-get install ros-melodic-map-server
```

8. (Do it each time that you work): Saved map

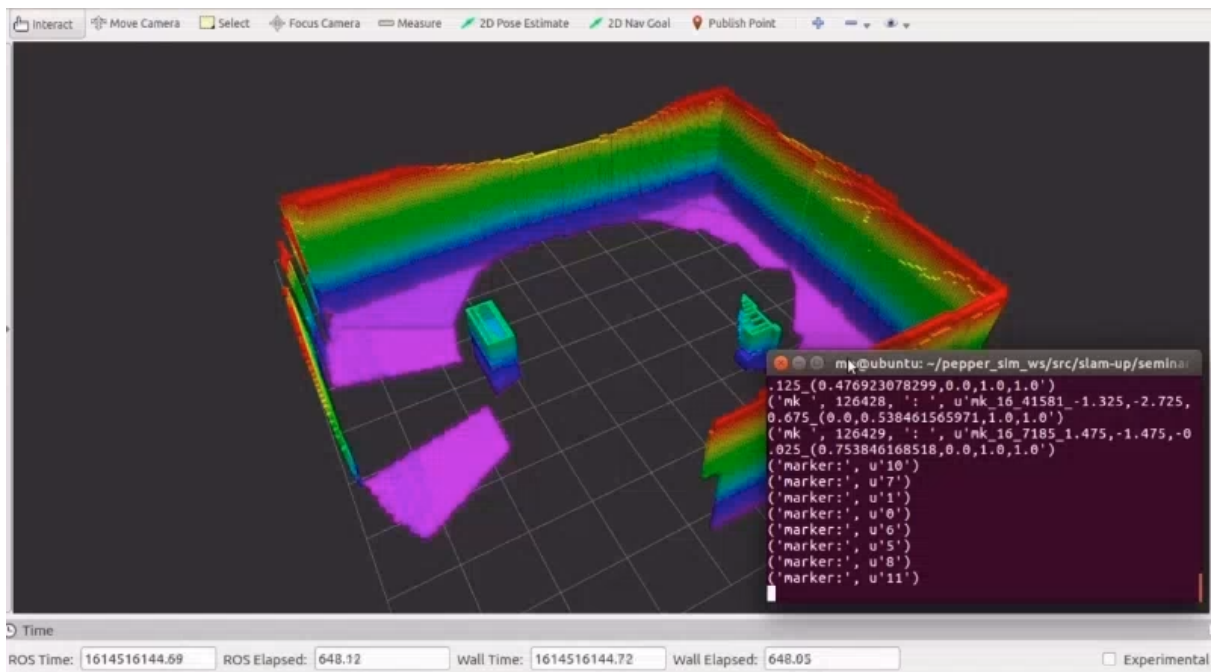
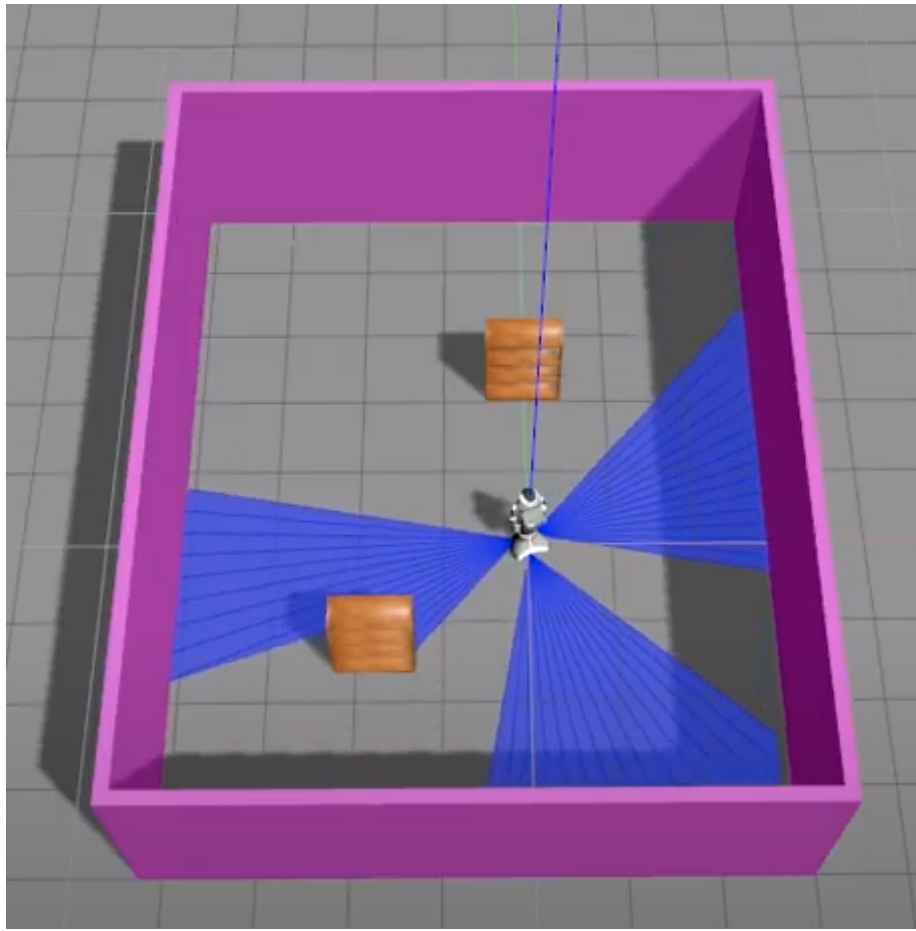
```
$roslaunch map_server map_saver -f 'file'
```

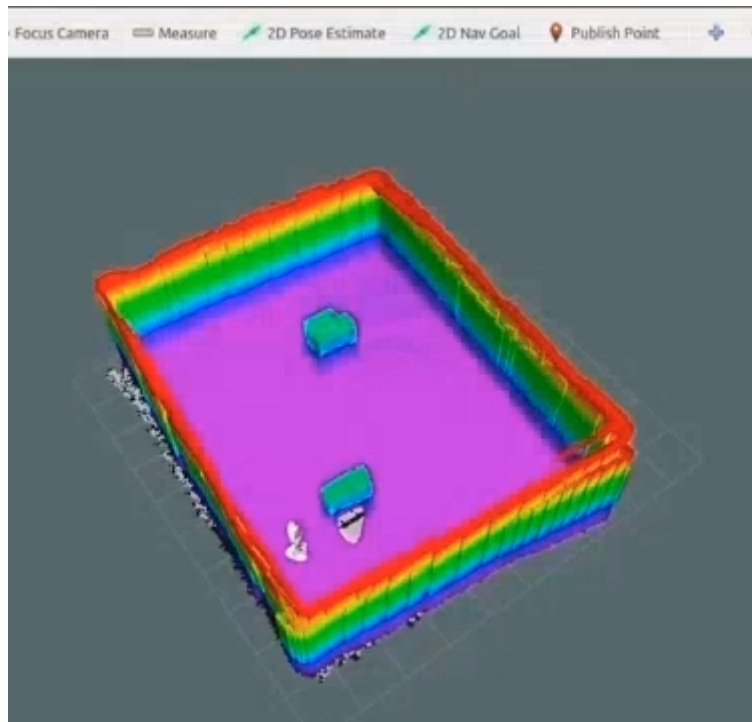
```
#“Ex. roslaunch map_server map_saver -f
```

```
'/home/jada/pepper_sim_ws/src/pepper_virtual/pepper_gazebo_plugin/map/map_2”
```



SLAM 3D:





Navigation

1. First download the corrected files for navigation. Click [here](#) to download.

2. Save and extract`sudo apt-get install ros-melodic-amcl` the zip files in the folder:

`~/pepper_sim_ws/src/pepper_virtual/`

3. We will return to the main folder directory and create a catkin space:

`$sudo apt-get install ros-melodic-move-base-msgs`

`$sudo apt-get install ros-melodic-move-base`

`$sudo apt-get install ros-melodic-navigation`

`$cd ~/pepper_sim_ws/`

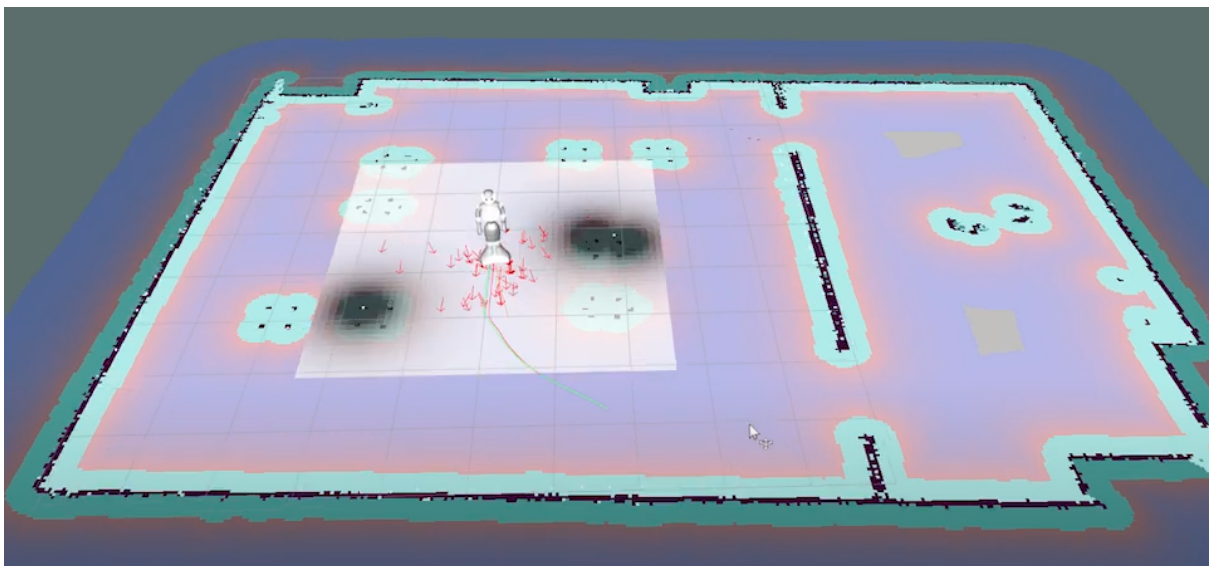
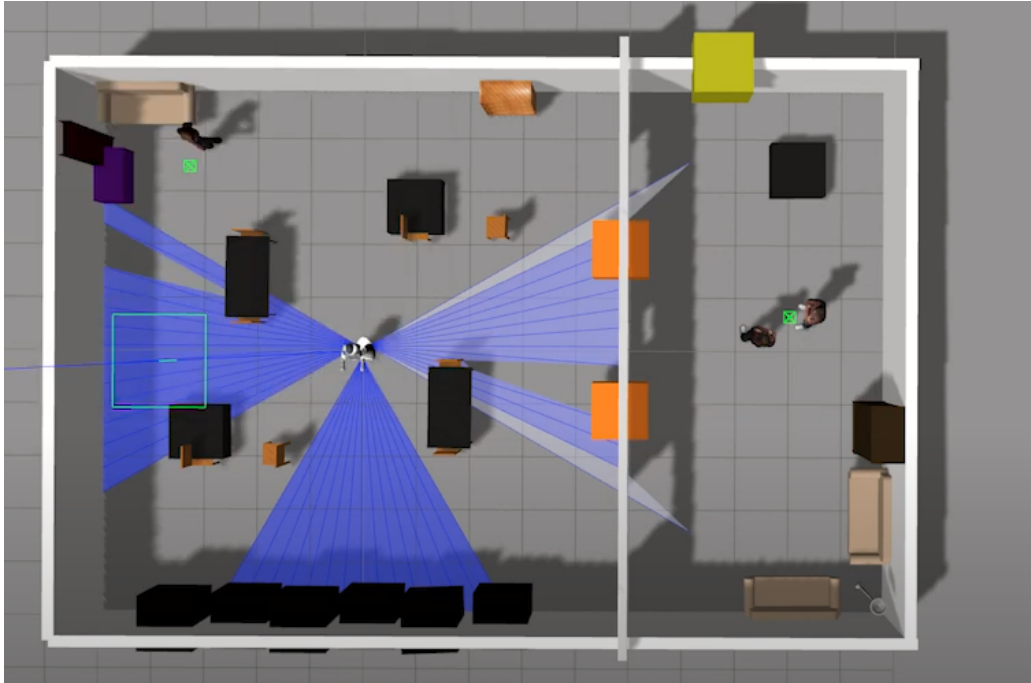
`$catkin_make`

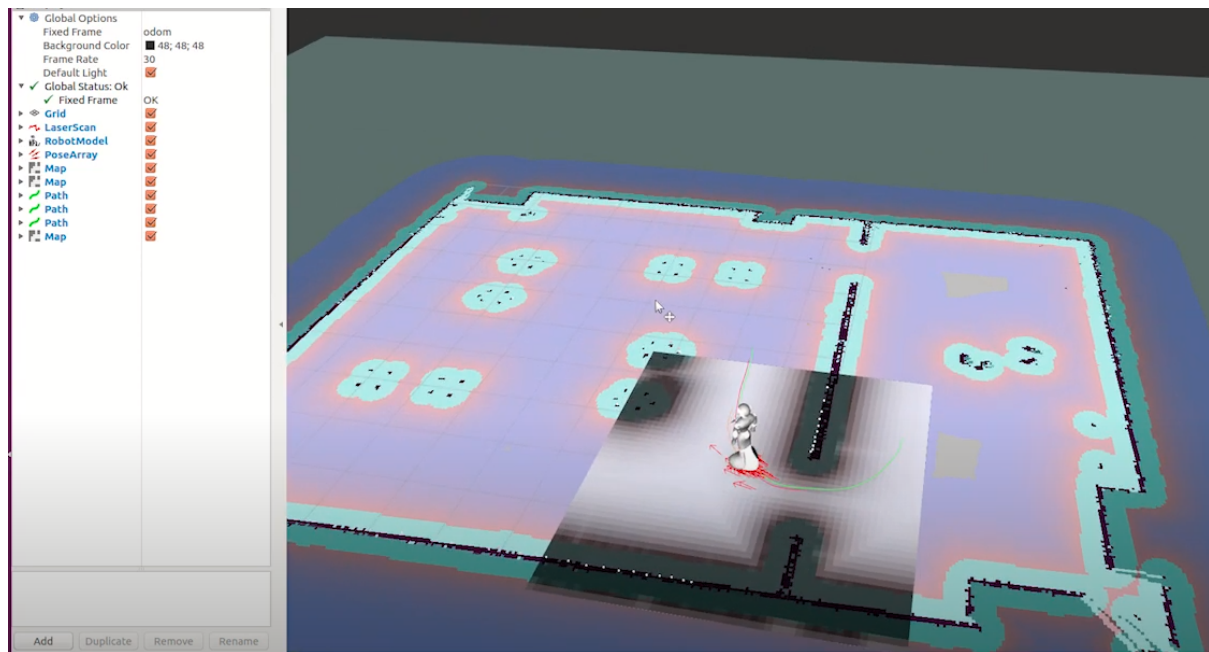
4. We will execute the line to initialize the robot on a map in the gazebo:

`$roslaunch pepper_nav restaurant_world.launch`

5. Now we will execute the line to do the navigation and open Rviz:

`$roslaunch pepper_nav amcl.launch`





Perception

Object detection with YOLO in ROS: YOLO is a real-time object detection system based on Deep Learning. There are several versions of YOLO, the version that is implemented in this tutorial is version 2 of YOLO. To use YOLO, Darknet is installed because it is an “official” implementation of YOLO.

Reference Website: https://github.com/Intelligent-Quads/iq_tutorials/blob/master/docs/intro_to_yolo.md

Note: You do not have to have anaconda installed, because you will have errors.

1. Installing dependencies only for ROS Melodic

```
$sudo apt-get install -y gcc-6
```

2. Creating the Darknet workspace in ~ /catkin_ws

```
$mkdir -p catkin_ws/src
```

```
$cd ~/catkin_ws/src/
```

3. Install darknet_ROS:

```
$git clone --recursive https://github.com/leggedrobotics/darknet_ros.git
```

4. Build Darknet:

```
$cd ~/catkin_ws/
```

For ROS Melodic:

```
$catkin build -DCMAKE_BUILD_TYPE=Release
```

For ROS Kinetic:

```
$catkin build -DCMAKE_BUILD_TYPE=Release-DCMAKE_C_COMPILER=/usr/bin/gcc-6
```

NOTE::

#If these commands to build Darknet do not work, one alternative is to use:

```
$catkin_make
```

5. Modify original bashrc:

```
$gedit ~/.bashrc
```

For Example: source /home/marco/catkin_ws/devel/setup.bash

```
$reboot
```

Darknet configuration

Change camera_reading topic from “/camera/rgb/image_raw” to “/pepper/camera/front/image_raw”

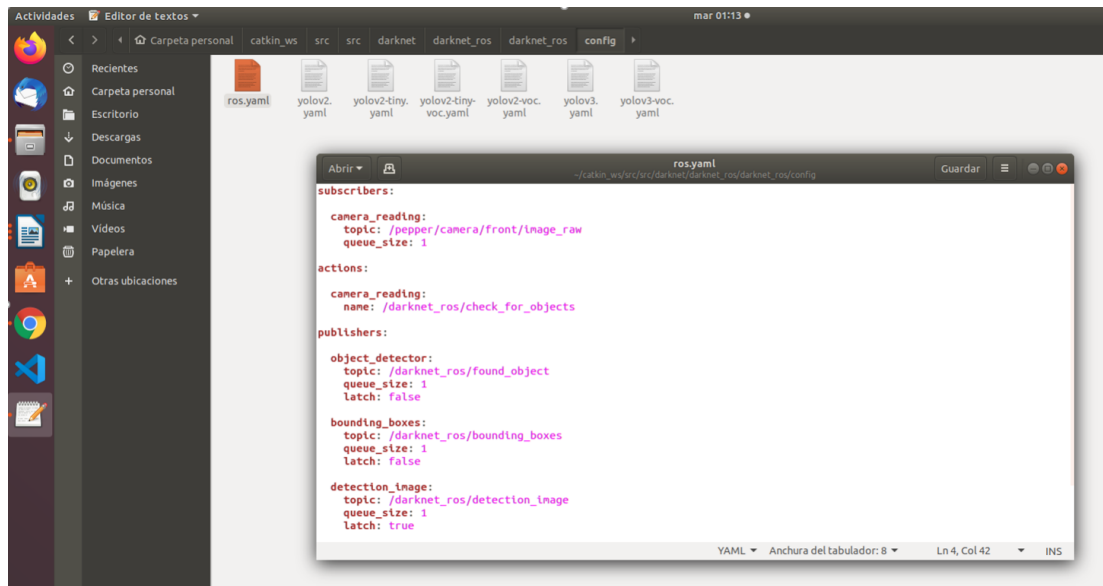


Fig: Darknet configuration.

Basic Operation:

\$roslaunch pepper_gazebo_plugin pepper_gazebo_plugin_in_office_CPU.launch

\$roslaunch darknet_ros darknet_ros.launch

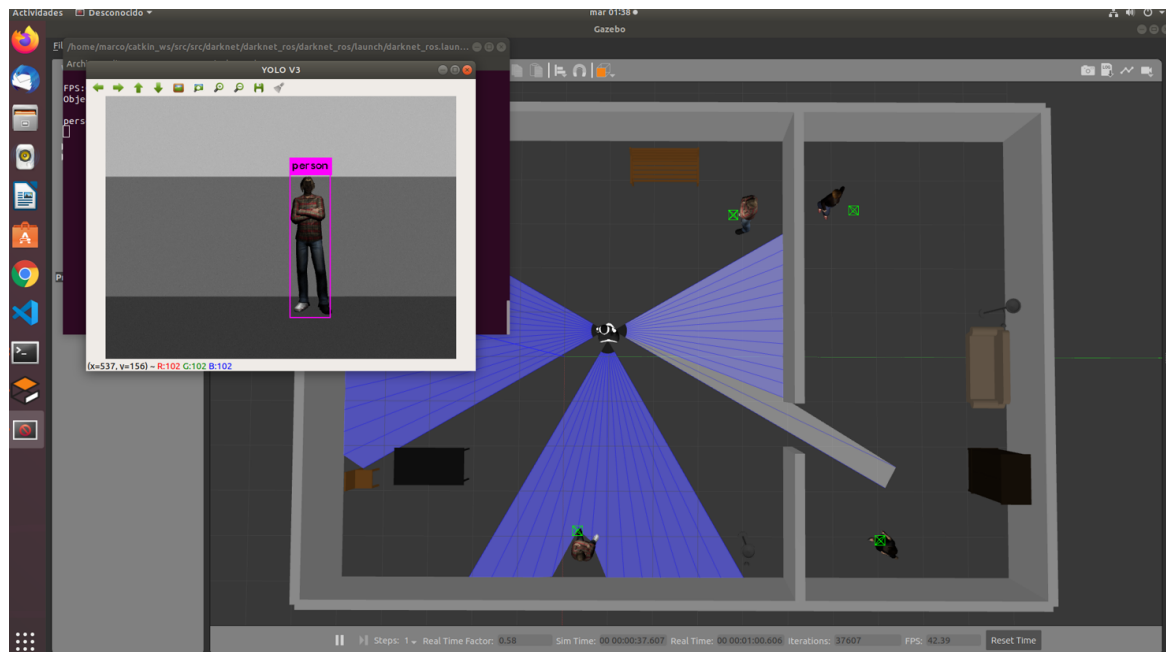


Fig: Detection of people with Yolo.

Pepper Robot

Technical Information:

http://doc.aldebaran.com/2-4/family/pepper_technical/motors_pep.html
